HEALTHPARTNERS | RESEARCH INSTITUTE | DEV DAYS FOLLOW-UP

Research Cohort Builder: getting started

From an investigator's code list to a governed cohort, an automated QA gate, and a de-identified analytic file you can export or share, with every number traceable to the code list that produced it.

1 run = 1 study
every study setting is a job parameter

1 count per step
an attrition funnel from the logic that ran

FAIL stops it
QA as code, before anything is exported

Tag once
identifiers masked everywhere outside the approved group

What we heard, and where it lives

The need	How the package addresses it
Repeatable study pulls	One parameterized **study** job: cohort, QA, analytic file, export. A new study is a new set of parameters and a code list, not a new script.
Investigator intake	Code lists (ICD-10-CM, NDC) and MRN lists are uploaded as **governed tables**, with an owner, history and lineage.
Quality before analysis	About 25 automated checks write `PASS`, `WARN` or `FAIL`; a `FAIL` stops the job. Small cells are flagged and suppressed.
Least-privilege access to PHI	**Governed tags** plus **ABAC column masks** GA: identifiers are masked for everyone outside an approved, account-level group.
Safe sharing	Only the de-identified view leaves: an audited volume export, an **OpenSharing** share, or a **Clean Room** when partners must not see row-level records.

How it fits together

Code list — CSV upload becomes a governed table → **01 Cohort** — index date, inclusions, exclusions, one count per step → **02 QA** — PASS / WARN / FAIL, gate → **03 Analytic file** — identified (masked) and de-identified → **Out** — volume export, share or clean room

INPUTS THE JOB READS (YOUR STANDARDIZED OR ENRICHED TABLES, OR THE SYNTHETIC SAMPLES)

Patients MRN, DOB, demographics + **Encounters** dates, types + **Diagnoses** ICD-10-CM + **Medications** orders + **Lab results** LOINC + **NDC crosswalk** owned by pharmacy informatics

Why the code list is a governed table

It defines the study

Change one code and you change the cohort, the counts and the results. So the code list gets the same treatment as any other data the study depends on.

One version of record
One table with an owner and a description, read directly by the cohort logic. No codes pasted into SQL, no two analysts with slightly different lists.

Reproducible
Delta history shows exactly which list produced a cohort, and time travel re-runs it as of the protocol date.

Lineage
Unity Catalog records that the cohort, the analytic file and the share all came from this list, so a change shows every downstream file it affects.

Checked like data
QA tests the list itself: codes with no hits in the source, NDCs that don't map to the crosswalk, MRNs that aren't in the patients table.

Reusable
A curated value set (for example the E11 family) can be owned once and reused across studies instead of rebuilt each time.

Controlled
Grants decide who may change a definition, and every read and change is audited. MRN lists are PHI and are tagged and masked like any identifier.

The access model

00_governance.py

Control	How it works
Tags	Notebooks tag identifier columns as they create them: `class.medical_record_number`, `class.date_of_birth`, plus `class.ethnicity` and `class.location` for review.
Masks	Two ABAC policies on the schema mask any tagged MRN (last 4 characters) or date of birth (empty) for everyone except `approved_group`. New tables are covered as soon as they are tagged.
Approved group	An **account-level** group, for example per study or per IRB approval. Adding or removing a person takes effect within a few minutes; no table changes.
Correct outputs	The job reads identifiers from the source patients table, so files are right whoever runs it. QA check `R03` reports whether the run identity is approved.
Linkage token	A SHA-256 hash of the MRN with a key from a secret scope. In production, use keyed tokens from your tokenization service.

Volume export
CSV and Parquet of the de-identified view, in a Unity Catalog volume. Downloads are audited.

OpenSharing
GA
For a named partner under a data use agreement: read-only and revocable. The share job never adds recipients.

Clean Rooms
GA
When partners must not see row-level records. Supported for HIPAA workloads on Azure with the compliance security profile.

IMPORTANT Review the de-identification rules (columns dropped, date precision, age bands, small-cell handling) with your privacy office for each study. This package is a starting pattern, not a certified de-identification method.

What's in the package

research-cohort-builder/

Path	What it is
`databricks.yml`	The bundle: variables and the dev, staging and prod targets. The only file you must edit.
`resources/research_cohort.yml`	The setup job, the study job (with every study setting as a parameter) and the QA dashboard.
`src/01_build_study_cohort.py`	Code list parsing, index date, inclusions and exclusions as separate steps, attrition funnel, tags.
`src/02_qa_checks.py`	Row counts, duplicates, null rates, date sanity, code coverage, MRN reconciliation, small cells, and the gate.
`src/03_analytic_file.py`, `04_share.py`	Person-level file, de-identified view, analytic QA, CSV and Parquet export, optional share.
`src/00_setup.py`, `00_governance.py`, `00_sample_data.py`	Schema and volume; mask functions and ABAC policies; synthetic sources, code list, crosswalk and MRN list.
`data/`, `docs/`, `scripts/`	The code list template; a Genie Code prompt; the dashboard generator.

Get running in dev

About 20 minutes

1. **Configure.** In `databricks.yml` under `targets.dev`, set the workspace host, your catalog, a SQL warehouse ID and, if you have one, the account-level `approved_group`.
2. **Deploy.**

```
databricks auth login --host https://<your-dev-workspace>.azuredatabricks.net
databricks bundle deploy -t dev
```

3. **Set up.** Schema, volume, policies and about 20,000 synthetic patients:

```
databricks bundle run -t dev research_setup
```

4. **Build the sample study.**

```
databricks bundle run -t dev research_study
```

Open the run, then `rs_attrition_t2d`, `rs_qa_results_t2d` and the **Research Cohort QA** dashboard.

5. **See the masks.** Query `rs_af_t2d`: MRNs show only their last 4 characters and dates of birth are empty. Join the approved group and query again.

> **YOU SHOULD SEE**
> In our test on the synthetic data, setup took about 3 minutes and the study about 5. The funnel ran 20,000 patients → 2,644 with a qualifying diagnosis → 2,636 adults → 2,152 with a prior encounter → **2,033** after exclusions, plus 924 on the study drug without a diagnosis (informational). QA: **0 FAIL**, with deliberate warnings such as two codes with no hits (E11.21, E11.42), 109 persons without full follow-up and small strata flagged. Identifiers read masked; the export held 2,033 rows with no identifiers.

Run a new study

Parameters, not code

1. **Upload the code list.** Catalog Explorer > your schema > Create > Table > Upload file. Columns `study_id, code_type, code_value, description`; `code_type` is `ICD10CM` (prefix match: `E11` matches every E11.x) or `NDC` (any format).
2. **Optional MRN list.** A table with an `mrn` column; tag it like the sample list.
3. **Run the study job with parameters**, from the job's *Run now with different parameters* or the CLI:

```
databricks bundle run -t dev research_study --params study_id=ckd2026,code_table=<catalog>.<schema>.ckd_codes,exclusion_icd10_prefixes=N18.6
```

4. **Read QA first.** `WARN` rows are findings to review with the investigator; any `FAIL` stops the job before anything is written or exported.

Parameter	Default	Meaning
`study_id`	`t2d`	Suffix for every output
`code_table`	`rs_study_codes`	The uploaded code list
`index_window_start` / `_end`	2024-01-01 / 2025-12-31	Window for the qualifying diagnosis; the earliest is the index date
`lookback_days`	365	At least one encounter this many days before the index date
`exclusion_icd10_prefixes`	`E10,N18.6`	Any of these on or before the index date excludes the person
`mrn_table`, `apply_mrn_restriction`	sample list, `false`	Restrict the cohort to an uploaded MRN list
`followup_days` and baseline windows	365, 365, 180	Analytic-file windows
`share_name`	empty	OpenSharing share for the de-identified copy

Use your own data

Data mapping

Point the source variables at your Standardized or Enriched tables (or views that rename columns), then set `create_sample_data` to `"false"`. With OMOP, the same pattern applies using concept sets in place of raw code lists.

Variable	Required columns
`patients_table`	`patient_id, mrn, date_of_birth, gender, race, ethnicity, insurance_payer_category, zip_code`
`encounters_table`	`encounter_id, patient_id, encounter_date, encounter_type`
`diagnoses_table`	`encounter_id, patient_id, icd10_code` (with the dot, for example `E11.9`)
`medications_table`	`patient_id, medication_name, ordered_datetime`
`lab_results_table`	`patient_id, loinc_code, result_value, resulted_datetime`
`ndc_crosswalk_table`	`ndc_11, medication_name, rxnorm_concept_id`; use the crosswalk your pharmacy informatics team owns

Staging and prod

CI/CD

dev
Each analyst's copy, synthetic data.

→

staging
Deployed by CI on merge to main, run by a service principal.

→

prod
Deployed on approval. Your sources, approved group and secret key.

1. Create a service principal per environment with access to the source tables, the catalog and serverless jobs.
2. Set the host, catalog, warehouse, `approved_group`, source tables and `linkage_secret_scope` for `staging` and `prod`.
3. In GitHub, add `DATABRICKS_HOST`, `DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET` to the `staging` and `prod` environments, with required reviewers on `prod`. The workflows skip themselves until `DATABRICKS_HOST` is set.

Our recommendations

From the session

1. Treat code lists as data
Upload them as governed tables; build a shared library of value sets with named owners.

2. One step, one count
Keep inclusions and exclusions as separate steps so the funnel is the logic that ran.

3. QA as code
Start from these checks, share them across studies, and let a FAIL stop the build.

4. Tag once, mask everywhere
Governed tags plus ABAC policies with an account-level approved group per study.

5. Only de-identified data leaves
Volume export, OpenSharing for named partners, Clean Rooms when rows must stay private.

6. Genie Code drafts, analysts review
Generate cohort SQL from a plain description, then keep the stepwise version as the record.

First 60 days

Suggested

WEEKS 1-2

Walkthrough
Deploy in dev with the analysts who will own it.

WEEKS 2-4

Governance design
Privacy, research compliance and identity owners agree on tags, groups and export approval.

WEEKS 3-5

Data mapping
Map Standardized and Enriched tables and the NDC crosswalk.

WEEKS 5-8

First real study
One approved protocol, a named research lead, a chosen export path.

Learn more

- Declarative Automation Bundles and job parameters
- Attribute-based access control (ABAC) and governed tags
- Data lineage and Data Classification
- OpenSharing and Clean Rooms
- Create a table from an uploaded file